Trabajo en Clase 04 — Docker con Spring Boot: Multi-Stage, Volúmenes y Redes

Integrantes

Jhonnatan David Hernandez Martinez - 2201492
Diego Alejandro Gómez Pedraza - 2211060
Luis Sebastián Mora Cañas - 2211554

Se usó el proyecto existente **DemoAcademicoApplication** (Spring Boot 3.2.5, Java 21) y se compararon tres formas de construir su imagen Docker. Entorno: Linux, Docker Engine 29.9.1, Maven, OpenJDK 21.

Actividades

1. Se validó el proyecto localmente: `mvn clean package` (con pruebas) y ejecución del `.jar`.
2. Se crearon `Dockerfile.bad`, `Dockerfile.good` y `Dockerfile.multi`, cambiando solo el nombre del jar por el de este proyecto (`demo-academico-0.0.1-SNAPSHOT.jar`).
3. Se construyó cada imagen con `--no-cache` para medir el tiempo real de build, se ejecutó cada una y se probó el endpoint.
4. Se inspeccionó el contenido de cada imagen con `docker exec` para ver qué herramientas y archivos quedaron dentro.
5. Se probó la persistencia con el volumen `datos-spring-lab` y la comunicación por nombre en la red `red-spring-lab`.

Nota sobre el puerto: el `application.yaml` del proyecto usa `server.port: 8081`, por eso los contenedores se ejecutaron con `-p 8080:8081`: el endpoint se sigue probando en `http://localhost:8080`.

Actividad 1 — Proyecto

```
$ find . -path ./target -prune -o -path ./.git -prune -o -path ./.idea -prune -o -path ./.mvn -prune -o
-path ./images -prune -o -type f \( -name '*.java' -o -name '*.yml' -o -name '*.yaml' -o -name 'pom.xml'
-o -name 'Dockerfile*' -o -name mvnw \) -print | sort
./docker-compose.yml
./Dockerfile
./Dockerfile.bad
./Dockerfile.good
./Dockerfile.multi
./mvnw
./pom.xml
./src/main/java/co/edu/demoacademico/config/DataSeeder.java
./src/main/java/co/edu/demoacademico/controller/EstudianteController.java
./src/main/java/co/edu/demoacademico/controller/GlobalExceptionHandler.java
./src/main/java/co/edu/demoacademico/DemoAcademicoApplication.java
./src/main/java/co/edu/demoacademico/model/Estudiante.java
./src/main/java/co/edu/demoacademico/repository/EstudianteRepository.java
./src/main/java/co/edu/demoacademico/service/EmailDuplicadoException.java
./src/main/java/co/edu/demoacademico/service/EstudianteNoEncontradoException.java
./src/main/java/co/edu/demoacademico/service/EstudianteService.java
./src/main/resources/application.yaml
./src/test/java/co/edu/demoacademico/DemoAcademicoApplicationTests.java
```

- **¿Proyecto existente o nuevo?** Existente: el módulo de Estudiantes de los laboratorios anteriores. Ya compila con Maven, genera un `.jar` y expone HTTP, que es lo que pide la opción A.
- **Endpoint:** `GET /api/estudiantes/pagina?page=0&size=2`, que devuelve estudiantes paginados en JSON (el seeder carga 500 al iniciar).
- **Nombre base de las imágenes:** `spring-docker`, con los sufijos `-bad`, `-good` y `-multi`.

Actividad 2 — Validación local

```
$ mvn clean package 2>&1 | grep -E 'Tests run|BUILD|Building jar|Total time' ; ls -lh target/*.jar
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.270 s -- in co.edu.demoacademic
o.DemoAcademicoApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building jar: /home/jhonatan/Documents/DemoAcademicoApplication/target/demo-academico-0.0.1-SNAPS
HOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  15.526 s
-rw-r--r-- 1 jhonatan jhonatan 53M Jan 31  1980 target/demo-academico-0.0.1-SNAPSHOT.jar

$ java -jar target/demo-academico-0.0.1-SNAPSHOT.jar   (y en otra terminal: curl http://localhost:8081/a
pi/estudiantes/pagina?page=0&size=2)
2026-10-18T10:42:07.535-05:00  INFO 8833 --- [DemoAcademicoApplication] [           main] o.s.b.w.embedd
ed.tomcat.TomcatWebServer  : Tomcat started on port 8081 (http) with context path ''
2026-10-18T10:42:07.557-05:00  INFO 8833 --- [DemoAcademicoApplication] [           main] c.e.d.DemoAcad
emicoApplication           : Started DemoAcademicoApplication in 6.454 seconds (process running
{"content":[{"id":1,"nombre":"Francisco Roybal Lozada","email":"est0_homero.rubio@yahoo.com"},{"id":2,"n
ombre":"Graciela Paz Medrano","email":"est1_martin.ramirez@yahoo.com"}],"pageable":{"pageNumber":0,"page
Size":2,"sort":{"sorted":false,"empty":true,"unsorted":true},"offset":0,"paged":true,"unpage
```

- **¿Por qué validar antes de contenerizar?** Si el proyecto falla localmente, Docker solo empaqueta el error y se vuelve más difícil saber si la falla es del código o del contenedor. En el Trabajo 03 esto ocurrió: el jar no arrancaba por un conflicto de SnakeYAML que había que corregir en el `pom.xml`, no en Docker.
- **¿Qué archivo genera Spring Boot?** Un *fat jar* ejecutable: `target/demo-academico-0.0.1-SNAPSHOT.jar` (53 MB), que ya incluye Tomcat y todas las dependencias.
- **¿Qué puerto usa?** El 8081, definido en `application.yaml`.

Actividad 3 — Primer enfoque: compilar dentro del Dockerfile

```
FROM maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn clean package
CMD ["java", "-jar", "target/demo-academico-0.0.1-SNAPSHOT.jar"]
```

```
$ docker build --no-cache -f Dockerfile.bad -t spring-docker-bad .
Step 1/5 : FROM maven:3.9-eclipse-temurin-21
Step 2/5 : WORKDIR /app
Step 3/5 : COPY . .
Step 4/5 : RUN mvn clean package
Step 5/5 : CMD ["java", "-jar", "target/demo-academico-0.0.1-SNAPSHOT.jar"]
Successfully built 29a50b2d33b9
Successfully tagged spring-docker-bad:latest
… (613 dependencias descargadas de Maven Central durante el build)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
Tiempo total del build: 66.2 s

$ docker run --rm -p 8080:8081 spring-docker-bad
2026-10-18T15:44:44.802Z  INFO 1 --- [DemoAcademicoApplication] [           main] o.s.b.w.embedded.tomca
t.TomcatWebServer  : Tomcat started on port 8081 (http) with context pa
2026-10-18T15:44:44.823Z  INFO 1 --- [DemoAcademicoApplication] [           main] c.e.d.DemoAcademicoApp
lication           : Started DemoAcademicoApplication in 6.747 seconds

$ curl 'http://localhost:8080/api/estudiantes/pagina?page=0&size=2'
{"content":[],"pageable":{"pageNumber":0,"pageSize":2,"sort":{"sorted":false,"empty":true,"unsorted":tru
e},"offset":0,"paged":true,"unpaged":false},"last":true,"totalElements":0,"totalPages":0,"first" …

$ docker exec lab-bad sh -c '…'   # inspección del contenido de la imagen
usuario: root
mvn: /usr/bin/mvn
javac: /opt/java/openjdk/bin/javac
ls /app:
diagrama.jpg docker-compose.yml Dockerfile Dockerfile.bad Dockerfile.good Dockerfile.multi .git .gitattr
ibutes .gitignore HELP.md .idea images Informe_03-Docker.pdf lab_2.pdf laboratorios .mvn mvnw mvnw.cmd p
om.xml src target Trabajo_04.html
repositorio Maven (~/.m2): 91M
código fuente en /app/src: sí
historial git en /app/.git: 1.2M
```

- **Herramientas de build que quedan en la imagen:** Maven (`/usr/bin/mvn`), el JDK completo con `javac` y el repositorio local de Maven `~/.m2` con 91 MB de dependencias descargadas.
- **Archivos adicionales:** `COPY . .` copió todo el directorio: código fuente, `.git` (1.2 MB de historial), `.idea`, los PDF de los laboratorios, imágenes, la carpeta `target` con clases intermedias e incluso los otros Dockerfiles. No hay `.dockerignore` que lo filtre.
- **Efecto sobre el tamaño:** la imagen pesa **736 MB**, contra 418 MB de los otros enfoques (76 % más). Comprimida para transferir pesa 415MB contra 231MB.
- **Por qué no es ideal para producción:** la imagen es más pesada de subir y descargar, expone el código fuente y el historial git, y deja herramientas como Maven y el compilador que un atacante podría usar dentro del contenedor. Además cada build vuelve a descargar las 613 dependencias, porque `COPY . .` invalida la caché antes de `mvn`.

Actividad 4 — Segundo enfoque: compilación separada

```
FROM eclipse-temurin:21-jdk-alpine
WORKDIR /app
COPY target/demo-academico-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
$ docker build --no-cache -f Dockerfile.good -t spring-docker-good .
Step 1/4 : FROM eclipse-temurin:21-jdk-alpine
Step 2/4 : WORKDIR /app
Step 3/4 : COPY target/demo-academico-0.0.1-SNAPSHOT.jar app.jar
Step 4/4 : ENTRYPOINT ["java", "-jar", "app.jar"]
Successfully built 8ec8afe4a9b4
Successfully tagged spring-docker-good:latest
Tiempo total del build: 3.0 s

$ docker run --rm -p 8080:8081 spring-docker-good
2026-10-18T15:44:52.689Z  INFO 1 --- [DemoAcademicoApplication] [           main] o.s.b.w.embedded.tomca
t.TomcatWebServer  : Tomcat started on port 8081 (http) with context pa
2026-10-18T15:44:52.708Z  INFO 1 --- [DemoAcademicoApplication] [           main] c.e.d.DemoAcademicoApp
lication           : Started DemoAcademicoApplication in 6.147 seconds

$ curl 'http://localhost:8080/api/estudiantes/pagina?page=0&size=2'
{"content":[{"id":1,"nombre":"Hugo Olivares Espinoza","email":"est0_jorge.ozuna@yahoo.com"},{"id":2,"nom
bre":"Margarita Patiño Báez","email":"est1_elisa.chacon@hotmail.com"}],"pageable":{"pageNumber …

$ docker exec lab-good sh -c '…'   # inspección del contenido de la imagen
usuario: root
mvn: no instalado
javac: /opt/java/openjdk/bin/javac
ls /app:
app.jar
repositorio Maven (~/.m2): no existe
código fuente en /app/src: no
historial git en /app/.git: no
```

- **¿Cuál imagen es más grande?** La de compilar dentro del Dockerfile: 736 MB contra 418 MB.
- **Ventaja de generar el `.jar` antes:** el build de la imagen tardó 3.0 s, porque solo copia un archivo. La compilación (15.5 s con pruebas) se hace una vez con la caché local de Maven, y la imagen contiene únicamente `app.jar`.
- **Separar build y runtime** significa que el entorno que compila (Maven, JDK, código fuente, dependencias de prueba) es distinto del entorno que ejecuta, que solo necesita una JVM y el jar.
- **Por qué mejora el despliegue:** la imagen es más pequeña (231MB comprimida), así que se sube y se descarga más rápido en cada servidor. Además el artefacto que se probó localmente es el mismo que se despliega. La desventaja es que depende de que el jar se compile antes en una máquina o pipeline con el JDK correcto.

Actividad 5 — Tercer enfoque: multi-stage

```
# Etapa 1: build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Etapa 2: runtime
FROM eclipse-temurin:21-jdk-alpine
WORKDIR /app
COPY --from=build /app/target/demo-academico-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
$ docker build --no-cache -f Dockerfile.multi -t spring-docker-multi .
Step 1/8 : FROM maven:3.9-eclipse-temurin-21 AS build
Step 2/8 : WORKDIR /app
Step 3/8 : COPY . .
Step 4/8 : RUN mvn clean package -DskipTests
Step 5/8 : FROM eclipse-temurin:21-jdk-alpine
Step 6/8 : WORKDIR /app
Step 7/8 : COPY --from=build /app/target/demo-academico-0.0.1-SNAPSHOT.jar app.jar
Step 8/8 : ENTRYPOINT ["java", "-jar", "app.jar"]
Successfully built c174f4e452f1
Successfully tagged spring-docker-multi:latest
… (598 dependencias descargadas de Maven Central durante el build)
[INFO] BUILD SUCCESS
Tiempo total del build: 56.0 s

$ docker run --rm -p 8080:8081 spring-docker-multi
2026-10-18T15:45:00.228Z  INFO 1 --- [DemoAcademicoApplication] [           main] o.s.b.w.embedded.tomca
t.TomcatWebServer  : Tomcat started on port 8081 (http) with context pa
2026-10-18T15:45:00.248Z  INFO 1 --- [DemoAcademicoApplication] [           main] c.e.d.DemoAcademicoApp
lication           : Started DemoAcademicoApplication in 5.779 seconds

$ curl 'http://localhost:8080/api/estudiantes/pagina?page=0&size=2'
{"content":[],"pageable":{"pageNumber":0,"pageSize":2,"sort":{"sorted":false,"unsorted":true,"empty":tru
e},"offset":0,"paged":true,"unpaged":false},"totalPages":0,"totalElements":0,"last":true,"number …

$ docker exec lab-multi sh -c '…'   # inspección del contenido de la imagen
usuario: root
mvn: no instalado
javac: /opt/java/openjdk/bin/javac
ls /app:
app.jar
repositorio Maven (~/.m2): no existe
código fuente en /app/src: no
historial git en /app/.git: no
```

- **Etapa build:** parte de la imagen de Maven, copia el proyecto, descarga las dependencias y ejecuta `mvn clean package -DskipTests` para producir el jar.
- **Etapa runtime:** parte de cero desde `eclipse-temurin:21-jdk-alpine`, que es mucho más liviana, y solo define el directorio de trabajo y el comando de arranque.
- **Qué se copia:** exactamente un archivo, `/app/target/demo-academico-0.0.1-SNAPSHOT.jar`, que en la imagen final se llama `app.jar` (`COPY --from=build`).
- **Qué no queda en la imagen final:** Maven, el repositorio `~/.m2`, el código fuente, `.git`, `.idea`, los PDF y la carpeta `target`. La inspección lo confirma: en `/app` solo hay `app.jar`.
- **Ventaja frente a una sola etapa:** el resultado es igual de pequeño y seguro que con la compilación separada (418 MB), pero la compilación sigue siendo reproducible dentro de Docker. Cualquier máquina con Docker construye la imagen sin tener Java ni Maven instalados.

Comparación de enfoques

```
$ docker images | grep -E 'REPOSITORY|spring-docker'
REPOSITORY              TAG                  IMAGE ID       CREATED              SIZE
spring-docker-multi     latest               c174f4e452f1   About a minute ago   418MB
spring-docker-good      latest               8ec8afe4a9b4   2 minutes ago        418MB
spring-docker-bad       latest               29a50b2d33b9   2 minutes ago        736MB
```

Criterio	Compila dentro del Dockerfile	Compilación separada	Multi-stage
Tamaño de imagen	736 MB (415MB comprimida)	418 MB (231MB comprimida)	418 MB (231MB comprimida)
Tiempo de build (sin caché)	66.2 s, descarga 613 dependencias y corre pruebas	3.0 s de imagen + 15.5 s de `mvn package` local	56.0 s, descarga 598 dependencias
Tiempo de despliegue	Arranque 8.3 s, pero transfiere casi el doble de datos	Arranque 7.6 s, transfiere menos	Arranque 7.2 s, transfiere menos
Seguridad	Baja: Maven, JDK, código fuente, `.git` y `~/.m2` dentro; corre como root	Mejor: solo `app.jar`; aún incluye JDK y corre como root	Mejor: solo `app.jar`; aún incluye JDK y corre como root
Adecuación para producción	No recomendable	Buena si existe un pipeline de CI que compile el jar	La más recomendable: pequeña y reproducible solo con Docker

El arranque es similar en los tres casos (unos 7–8 s) porque la JVM ejecuta el mismo jar. La diferencia en despliegue está en cuántos datos hay que transferir a cada servidor y en cuánto tarda el build.

Actividad 6 — Análisis de la tabla

- **Tamaño al compilar dentro del Dockerfile:** crece porque cada capa queda guardada en la imagen. Las de `COPY . .` y `RUN mvn` conservan herramientas, dependencias y archivos que la aplicación no usa para ejecutarse. En este proyecto eso fueron 318 MB extra.
- **Ventajas de separar compilación y ejecución:** imágenes más pequeñas, builds de imagen más rápidos, menos archivos expuestos y un único artefacto (el jar) que se prueba una vez y se despliega igual en todos los ambientes.
- **Impacto en seguridad y despliegue:** con menos software dentro de la imagen hay menos vulnerabilidades posibles y menos herramientas para un atacante, y no se filtra el código fuente ni el historial. En despliegue se transfiere casi la mitad de datos, por lo que escalar o restaurar un servicio es más rápido.

Actividad 7 — Persistencia con volúmenes

```
$ docker volume create datos-spring-lab && docker run --rm -v datos-spring-lab:/datos alpine sh -c "echo 'Persistencia Spring Boot' > /datos/info.txt" && docker run --rm -v datos-spring-lab:/datos alpine cat /datos/info.txt && docker volume ls | grep datos-spring-lab
datos-spring-lab
Persistencia Spring Boot
local     datos-spring-lab

$ docker run --rm alpine sh -c "echo 'sin volumen' > /tmp/info.txt"; docker run --rm alpine cat /tmp/info.txt
cat: can't open '/tmp/info.txt': No such file or directory
```

El archivo escrito en el volumen sobrevive al contenedor que lo creó. El mismo experimento sin volumen (segundo comando) pierde el archivo apenas el contenedor termina.

- **Qué resuelven los volúmenes:** separan los datos del ciclo de vida del contenedor. Los contenedores son desechables; los datos no.
- **Base de datos sin volumen:** todos los registros vivirían en la capa de escritura del contenedor y se perderían al eliminarlo, al actualizar la imagen o al recrearlo con `docker compose down`/`up`.
- **Importancia empresarial:** en una empresa los datos (clientes, transacciones, notas) son el activo más valioso. Los volúmenes permiten actualizar o reemplazar contenedores sin perder información y además facilitan los respaldos.

Actividad 8 — Comunicación con redes

```
$ docker network create red-spring-lab && docker run -d --name db-lab --network red-spring-lab -e MYSQL_
ROOT_PASSWORD=123456 -e MYSQL_DATABASE=demo mysql:8.0 && docker run --rm --network red-spring-lab alpine
sh -c 'ping -c 2 db-lab'
6954f31e0fddcda897de8ae4f40e165e15edc5446d314613cf2794144f3bf85e
c475307cd6dcc1dee248973a3480b2b84b2d306c98033549e204b3bb0aee211a
PING db-lab (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.094 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.066 ms

--- db-lab ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.066/0.080/0.094 ms

$ docker network inspect red-spring-lab --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}{{en
d}}'
db-lab 172.18.0.2/16

$ docker run --rm alpine sh -c 'ping -c 2 db-lab'
ping: bad address 'db-lab'
```

Dentro de `red-spring-lab` el nombre `db-lab` se resolvió a 172.18.0.2 y respondió sin pérdida de paquetes. El mismo ping desde un contenedor fuera de la red falla con `bad address`: el DNS interno de Docker solo funciona entre contenedores de la misma red definida por el usuario.

- **Para qué sirven las redes:** conectan contenedores entre sí de forma aislada del resto, con resolución de nombres automática.
- **Por qué Spring Boot las necesita:** la aplicación y la base de datos están en contenedores distintos. Para conectarse, el `datasource` necesita una dirección, por ejemplo `jdbc:mysql://db-lab:3306/demo`, y la red es lo que hace que esa dirección exista.
- **Ventaja de usar el nombre:** la IP de un contenedor cambia cada vez que se recrea, pero el nombre se mantiene. La configuración no tiene IPs fijas y funciona igual en cualquier máquina.

Actividad 9 — Caso integrador

- **Errores en la construcción de sus imágenes:** compilan dentro de una sola etapa con Maven, copian todo el proyecto sin `.dockerignore` y usan imágenes base pesadas. Por eso las imágenes son grandes y cada build descarga de nuevo las dependencias.
- **Cambio en build y runtime:** separar las dos fases y que la imagen final contenga solo una JVM y el jar.
- **¿Compilación separada o multi-stage?** Multi-stage. Da el mismo tamaño final (418 MB contra 736 MB en este laboratorio) sin depender de que cada desarrollador o servidor tenga el JDK y Maven correctos, así que la construcción es reproducible. Se complementa con un `.dockerignore` y copiando primero el `pom.xml` para aprovechar la caché de dependencias.
- **Papel de los volúmenes:** resolver la pérdida de información, montando los datos de la base de datos (por ejemplo `/var/lib/mysql`) en un volumen.
- **Papel de las redes:** resolver la falta de comunicación, poniendo los servicios en una red definida por el usuario (o en Docker Compose, que la crea automáticamente) y conectándolos por nombre de servicio, no por IP.
- **Mejoras esperadas:** en *tamaño*, cerca de 43 % menos; en *seguridad*, sin código fuente ni herramientas de build, y se podría ir más lejos con una imagen JRE y un usuario no root; en *persistencia*, datos que sobreviven a los contenedores; en *despliegue*, menos datos que transferir y builds reproducibles.

Preguntas guía de reflexión

1. **¿Por qué no compilar en el Dockerfile?** Sí se puede, pero no en la misma etapa que ejecuta. Si se hace en una sola etapa, la imagen final hereda Maven, dependencias y código fuente que no necesita.
2. **¿Qué pasa con el tamaño?** Aumenta: 736 MB contra 418 MB en este proyecto.
3. **¿Ventajas de separar compilación y ejecución?** Imágenes pequeñas, menos superficie de ataque, un artefacto único y probado, y despliegues más rápidos.
4. **¿Impacto en seguridad y despliegue en producción?** Menos software significa menos vulnerabilidades y no expone el código. Imágenes más livianas permiten escalar y recuperarse más rápido.
5. **¿Qué problema resuelven los volúmenes?** La pérdida de datos al eliminar o recrear contenedores.
6. **¿Qué función cumplen las redes?** Permiten que los contenedores se comuniquen de forma aislada y se encuentren por nombre.

Conclusión individual

Este laboratorio mostró que una aplicación que “funciona” en Docker no necesariamente está bien contenerizada. Las tres imágenes ejecutaron el mismo DemoAcademico y respondieron igual, pero la que compila dentro del Dockerfile pesó 736 MB frente a 418 MB y llevó dentro Maven, el código fuente, el historial de git y 91 MB de dependencias que nadie necesita en producción. Esa diferencia se paga en cada despliegue, porque hay que transferir casi el doble de datos, y en seguridad, porque cada herramienta extra es una posible vulnerabilidad. El enfoque multi-stage me parece el más adecuado: conserva la reproducibilidad de compilar dentro de Docker y entrega una imagen final con solo el jar. También quedó claro que la imagen es solo una parte: sin volúmenes los datos desaparecen con el contenedor, como se vio en la prueba sin volumen, y sin redes los servicios no se encuentran entre sí, como se vio en el ping fallido fuera de la red. Como mejora futura, usaría una imagen base JRE en vez de JDK, un usuario no root y un `.dockerignore` para seguir reduciendo tamaño y riesgo.